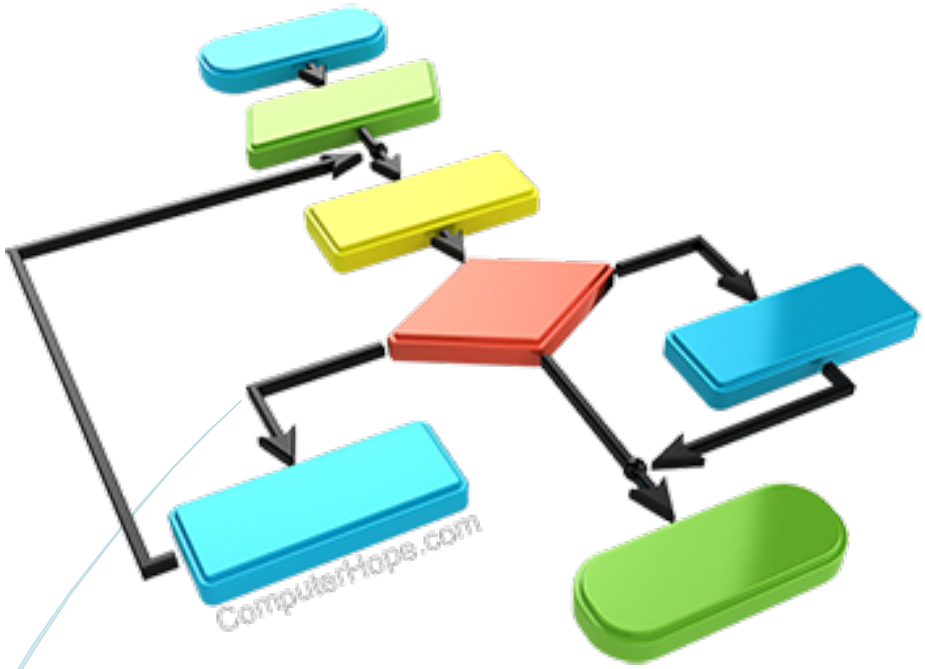
ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

25/6/2022

PROJECT ΣΤΟ ΜΑΘΗΜΑ:

"ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ"



Κυριακάτου Νίκη Αικατερίνη ΑΜ:1084624

Ζυγούρης Φίλιππος-Παρασκευάς ΑΜ:1084660

Ζάννος Εμμανουήλ-Λουκάς ΑΜ:1084543

Καλανδράκης Βασίλειος ΑΜ:1084577

ΠΕΡΙΕΧΟΜΕΝΑ:

ΜΕΡΟΣ 1

ΜΕΡΟΣ 2

ΜΕΡΟΣ 1

ΕΡΩΤΗΜΑ 1

Κώδικας που υλοποιήθηκε:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>
```

```
#include <sys\timeb.h>



typedef struct dedomena

{
char date[11];
float temperature;
} mydata;


int main()



{
mydata array[1406];
FILE *fp;
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
```

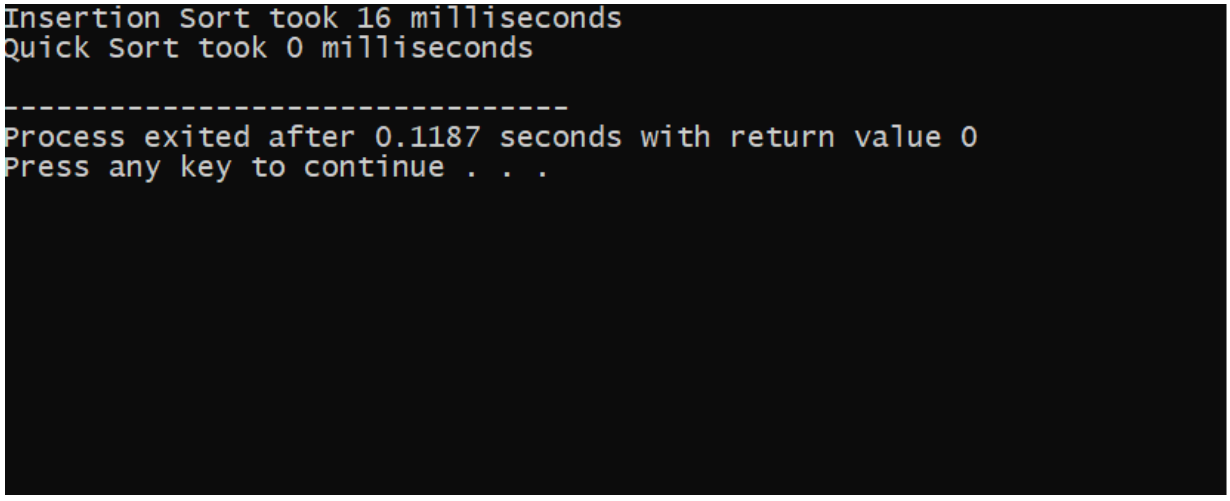
```
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
struct timeb start, end;
int diff;

ftime(&start);

insertionSort(array,1406);
ftime(&end);
diff = (int) (1000.0 * (end.time - start.time) + (end.millitm - start.millitm));

printf("Insertion Sort took %u milliseconds\n", diff);

ftime(&start);

quicksort(array,1406);
ftime(&end);
diff = (int)(1000.0 * (end.time - start.time) + (end.millitm - start.millitm));
printf("Quick Sort took %u milliseconds\n", diff);
fclose(fp);
}
```

```
void insertionSort(mydata *a, int length) {

int j;

int i;

mydata temp;

for (i = 1; i < length; i++) {

j = i;

while (j > 0 && a[j].temperature < a[j - 1].temperature || (a[j].temperature == a[j -
1].temperature && strcmp(a[j].date,a[j-1].date) < 0)) {

temp = a[j];

a[j] = a[j-1];

a[j-1] = temp;

j--;

}

}


}



void swap_data(mydata *left, mydata *right)

{

mydata tmp = *right;

*right = *left;

*left = tmp;

}
```

```
int compare_data(const mydata* left,
const mydata* right)
{
float therm = left->temperature - right->temperature;
return (therm ? therm : (strcmp(left->date,right->date)));
}

static void quicksort_(mydata *arr, int left, int right)
{
mydata p = arr[(left+right)/2];
int l = left, r = right;

while (l <= r)
{
while (compare_data(arr+l, &p) < 0)
++l;
while (compare_data(arr+r, &p) > 0)
--r;
if (l <= r)
{
swap_data(arr+l, arr+r);
++l; --r;
}
}

if (left < r)
```

```
quicksort_(arr, left, r);
if (l < right)
quicksort_(arr, l, right);
}

void quicksort(mydata *arr, int count)
{
if (arr && (count>0))
quicksort_(arr, 0, count-1);
}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:



Παρατηρούμε στην περίπτωση αυτή ότι ο Insertion Sort χρειάστηκε περισσότερο χρόνο σε σχέση με τον QuickSort (κρατήσαμε τις ακεραίες τιμές των milliseconds, για αυτό στην περίπτωση του QuickSort παρατηρούμε μηδενική τιμή).

ΕΡΩΤΗΜΑ 2

Κώδικας που υλοποιήθηκε:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>

#include <sys\timeb.h>



typedef struct dedomena

{
char date[11];
float phosphate;
} mydata;


int main()
```

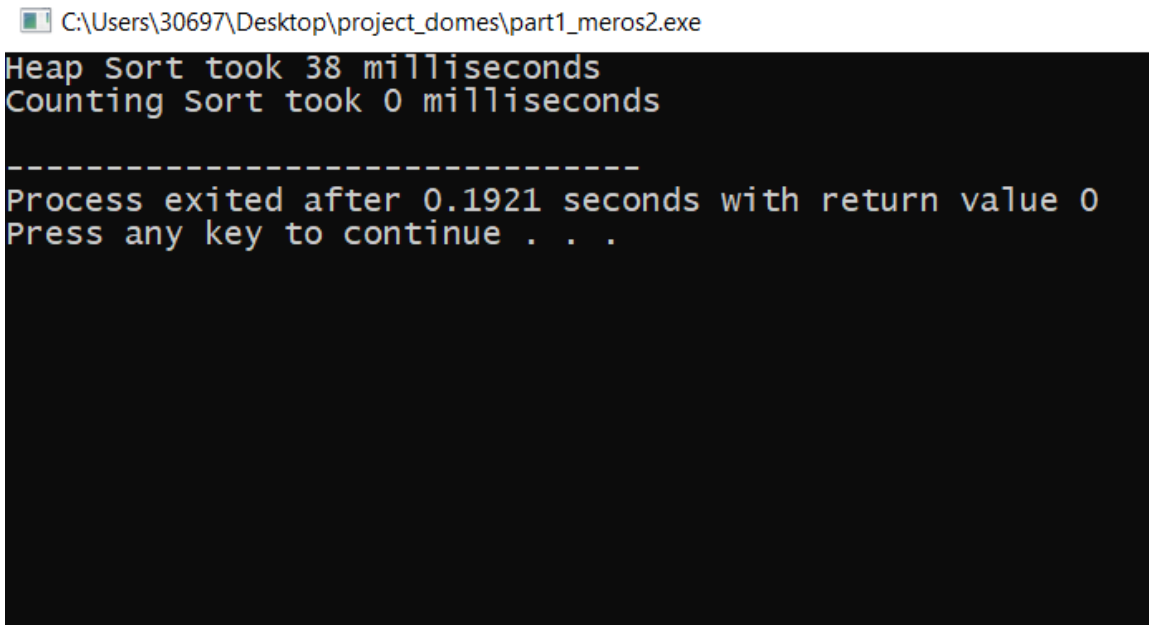
```
{
mydata array[1406];
FILE *fp;
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
token = strtok(NULL, ",");

array[count].phosphate = atof(token);
count++;

}
struct timeb start, end;
int diff;

ftime(&start);

heap_sort(array,1406);
```

```
ftime(&end);
diff = (int) (1000.0 * (end.time - start.time) + (end.millitm - start.millitm));

printf("Heap Sort took %u milliseconds\n", diff);

ftime(&start);

countSort(array, 1406) ;
ftime(&end);
diff = (int)(1000.0 * (end.time - start.time) + (end.millitm - start.millitm));
printf("Counting Sort took %u milliseconds\n", diff);
fclose(fp);
}



void swap(mydata* a,mydata* b){
mydata temp_data = *a;
*a = *b;
*b = temp_data;

return ;
}
void max_heapify(mydata arr[],int len_arr){
int i;
for(i=len_arr-1;i>0;i--){
mydata current_node_data = arr[i];
```

```
int father_node_idx = (int)((i-1)/2);

mydata father_node_data = arr[father_node_idx];

if(current_node_data.phosphate>father_node_data.phosphate ||
(current_node_data.phosphate == father_node_data.phosphate &&
strcmp(current_node_data.date,father_node_data.date) > 0)){

swap(&(arr[i]),&(arr[father_node_idx]));

}

}

swap(&(arr[0]),&(arr[len_arr-1]));

}


void heap_sort(mydata arr[],int len_arr){

int i;

for(i=len_arr;i>1;i--){

max_heapify(arr,i);

}

}



float getMax(mydata a[], int n) {

float max = a[0].phosphate;

int i;

for(i = 1; i<n; i++) {

if(a[i].phosphate > max)

max = a[i].phosphate;

}

return max;
```

```
}
void countSort(mydata a[], int n)
{
mydata output[n+1];
int max = (int)getMax(a, n);
int count[max+1];
int i;
for (i = 0; i <= max; ++i)
{
count[i] = 0;
}
for (i = 0; i < n; i++)
{
count[(int)a[i].phosphate]++;
}
for(i = 1; i<=max; i++)
count[i] += count[i-1];
for (i = n - 1; i >= 0; i--) {
output[(int)count[(int)a[i].phosphate] - 1] = a[i];
count[(int)a[i].phosphate]--;
}
for(i = 0; i<n; i++) {
a[i] = output[i];
}
}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:



Παρατηρούμε ότι ο Heapsort απαίτησε περισσότερο χρόνο σε σχέση με τον Counting Sort, όπως φαίνεται από το παραπάνω στιγμιότυπο εκτέλεσης του κώδικα.

ΕΡΩΤΗΜΑ 3

Κώδικας που υλοποιήθηκε:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>

#include <sys\timeb.h>



typedef struct dedomena

{
char date[11];
float temperature;
} mydata;


int main()
```

```
{
mydata array[1406];
FILE *fp;
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
char wanted[11];
printf("Please Give the date:");
scanf("%s",wanted);
int position;
struct timeb start, end;
int diff;
```

```
ftime(&start);


position = binarySearch(array,wanted,1406);
printf("Temperature found from Binary Search:%f\n",array[position].temperature);
ftime(&end);
diff = (int) (1000.0 * (end.time - start.time) + (end.millitm - start.millitm));


printf("BinarySeach took %u milliseconds\n", diff);


ftime(&start);


position = interpolationSearch(array,1406 ,wanted);
printf("Temperature found from Interpolation
Search:%f\n",array[position].temperature);
ftime(&end);
diff = (int) (1000.0 * (end.time - start.time) + (end.millitm - start.millitm));
printf("Interpolation Search took %u milliseconds\n", diff);




fclose(fp);
}


int binarySearch(mydata a[], char key[11],int n)
{
int low, high, mid;
low=0;
```

```
high=n-1;
while(low<=high)
{
mid=(low+high)/2;
if (strcmp(key,a[mid].date)==0)
{
return mid;
}
else if(strcmp(key,a[mid].date)>0)
{
high=high;
low=mid+1;
}
else
{
low=low;
high=mid-1;
}
}
return -1;
}



int interpolationSearch(mydata A[], int n, char x[11])
{
```

```
int low = 0, high = n - 1, mid;


while (strcmp(A[low].date,A[high].date)!=0 && strcmp(x,A[low].date) >=0 &&
strcmp(x,A[high].date)>=0)

{

mid = low + (strcmp(x,A[low].date) * (high - low) /
strcmp(A[high].date,A[low].date));

mid = roundf(mid);


if (strcmp(x,A[mid].date)==0)

return mid;


else if (strcmp(x,A[mid].date)<0)

high = mid - 1;

else

low = mid + 1;

}


if (strcmp(x,A[low].date)==0)

return low ;


else

return -1;

}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:

```
Please Give the date:01/07/2000
Temperature found from Binary Search:11.000000
BinarySeach took 0 milliseconds
Temperature found from Interpolation Search:11.000000
Interpolation Search took 0 milliseconds

--------------------------------
Process exited after 19.49 seconds with return value 0
Press any key to continue . . .
```

ΕΡΩΤΗΜΑ 4

Κώδικας που υλοποιήθηκε:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>

#include <math.h>



typedef struct dedomena

{
char date[11];
float temperature;
} mydata;



int main()
```

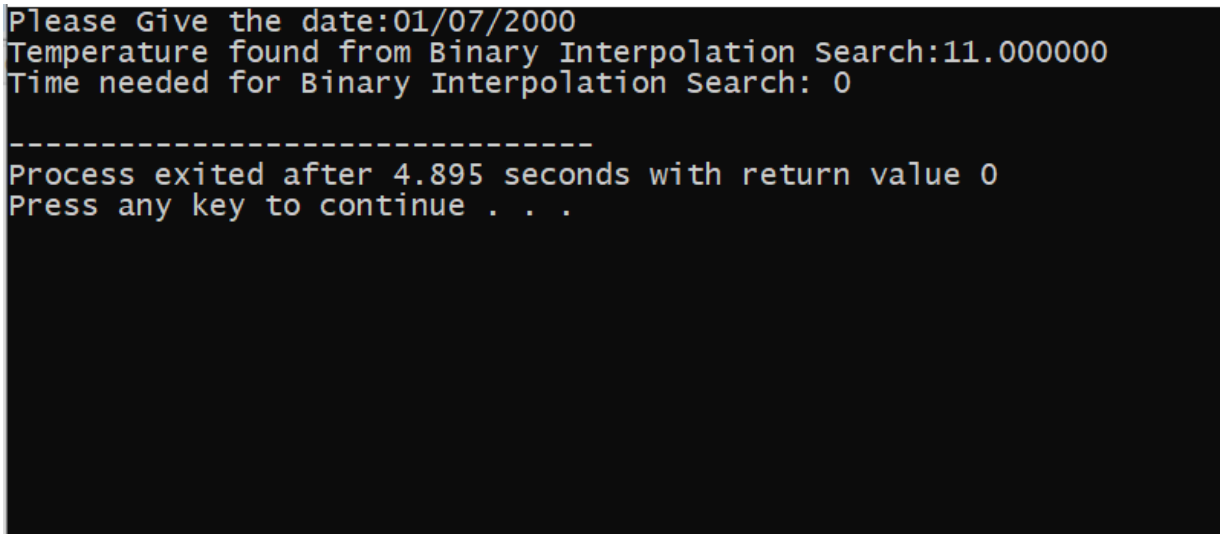
```
{
mydata array[1406];
FILE *fp;
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
char wanted[11];
printf("Please Give the date:");
scanf("%s",wanted);
int position;
clock_t start;
clock_t end;

start = clock();
position = binary_interpolation(array,0, 1405, wanted);
```

```
printf("Temperature found from Binary Interpolation
Search:%f\n",array[position].temperature);

end = clock();

double time_spent;

time_spent = (double)(end - start) / CLOCKS_PER_SEC;

printf("Time needed for Binary Interpolation Search: %d\n", time_spent);



fclose(fp);

}



int linearSearch (mydata *list, int l, int r, char key[11]) {

int found = 0, i = l;

while (!found && i < r) {

if (strcmp(list[i].date,key)==0) return i;

++i;

}

return -1;

}



int binary_interpolation (mydata *arr, int l, int r, char key[11]) {

int left = l;

int right = r;


int size = right - left;
```

```
int next = (int) (size *
((strcmp(key,arr[left].date)))/(strcmp(arr[right].date,arr[left].date))) + 1;


while (strcmp(key,arr[next].date)!=0) {

int i = 0;

size = right - left;

if (size <= 3)

return linearSearch (arr, l, r, key);

if (strcmp(key,arr[next].date)>=0) {

while (strcmp(key,arr[next + i*((int) sqrt(size)) - 1].date)>0) {

++i;

}

right = next + (int) (i*sqrt(size));

left = next + (int) ((i-1)*sqrt(size));

}

else if (strcmp(key,arr[next].date)<0) {

while (strcmp(key,arr[next -i* ((int) sqrt(size)) + 1].date)<0) {

++i;

}

right = next - (int) ((i-1)*sqrt(size));

left = next - (int) (i*sqrt(size));

}

next = (int) (left + ((right - left
+1)*(strcmp(key,arr[left].date))/(strcmp(arr[right].date,arr[left].date)))) - 1;

}

if (strcmp(key,arr[next].date)==0)

return next;
```

```
else return -1;

}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:



ΜΕΡΟΣ 2

ΕΡΩΤΗΜΑ Α

Κώδικας που υλοποιήθηκε:

```
#include<stdio.h>
#include<stdlib.h>
#include <string.h>

typedef struct dedomena

{
char date[11];
float temperature;
} mydata;
struct Node
{
char date[11];
float temperature;
struct Node *left;
struct Node *right;
int height;
};

int max(int a, int b);
```

```
int height(struct Node *N)
{
if (N == NULL)
return 0;
return N->height;
}

int max(int a, int b)
{
return (a>b)? a : b;
}

struct Node* newNode(char imerominia[11], float thermokrasia)
{
struct Node* node = (struct Node*)malloc(sizeof(struct Node));
strcpy(node->date,imerominia);
node->temperature = thermokrasia;
node->left = NULL;
node->right = NULL;
node->height = 1;
return(node);
}

struct Node *rightRotate(struct Node *y)
{
struct Node *x = y->left;
```

```
struct Node *T2 = x->right;
x->right = y;
y->left = T2;

y->height = max(height(y->left), height(y->right))+1;
x->height = max(height(x->left), height(x->right))+1;
return x;
}

struct Node *leftRotate(struct Node *x)
{
struct Node *y = x->right;
struct Node *T2 = y->left;
y->left = x;
x->right = T2;
x->height = max(height(x->left), height(x->right))+1;
y->height = max(height(y->left), height(y->right))+1;
return y;
}
int getBalance(struct Node *N)
{
if (N == NULL)
return 0;
return height(N->left) - height(N->right);
}
```

```
struct Node* insert(struct Node* node, char imerominia[11], float thermokrasia)
{
if (node == NULL)
return(newNode(imerominia,thermokrasia));
if (strcmp(imerominia,node->date)<0)
node->left = insert(node->left, imerominia,thermokrasia);
else if (strcmp(imerominia,node->date)>0)
node->right = insert(node->right, imerominia,thermokrasia);
else
return node;
node->height = 1 + max(height(node->left),
height(node->right));

int balance = getBalance(node);
if (balance > 1 && strcmp(imerominia, node->left->date)<0)
return rightRotate(node);
if (balance < -1 && strcmp(imerominia, node->right->date)>0)
return leftRotate(node);
if (balance > 1 && strcmp(imerominia, node->left->date)>0)
{
node->left = leftRotate(node->left);
return rightRotate(node);
}
if (balance < -1 && strcmp(imerominia, node->right->date)<0)
{
node->right = rightRotate(node->right);
```

```
return leftRotate(node);
}
return node;
}

struct Node * minValueNode(struct Node* node)
{
struct Node* current = node;
while (current->left != NULL)
current = current->left;
return current;
}
struct Node* deleteNode(struct Node* root, char imerominia[11])
{
if (root == NULL)
return root;
if (strcmp(imerominia,root->date)<0)
root->left = deleteNode(root->left, imerominia);
else if( strcmp(imerominia,root->date)>0 )
root->right = deleteNode(root->right, imerominia);

else
{
if( (root->left == NULL) || (root->right == NULL) )
{
struct Node *temp = root->left ? root->left :
```

```
root->right;
if (temp == NULL)
{
temp = root;
root = NULL;
}
else
*root = *temp;
free(temp);
}
else
{
struct Node* temp = minValueNode(root->right);
strcpy(root->date,temp->date);
root->temperature = temp->temperature;
root->right = deleteNode(root->right, temp->date);
}
}
if (root == NULL)
return root;
root->height = 1 + max(height(root->left),
height(root->right));
int balance = getBalance(root);
if (balance > 1 && getBalance(root->left) >= 0)
return rightRotate(root);
if (balance > 1 && getBalance(root->left) < 0)
```

```
{
root->left = leftRotate(root->left);
return rightRotate(root);
}
if (balance < -1 && getBalance(root->right) <= 0)
return leftRotate(root);
if (balance < -1 && getBalance(root->right) > 0)
{
root->right = rightRotate(root->right);
return leftRotate(root);
}
return root;
}
void printInorder(struct Node* node)
{
if (node == NULL)
return;
printInorder(node->left);
printf("---Date: %s Temperature: %f ---\n", node->date, node->temperature);
printInorder(node->right);
}

struct Node* search(struct Node* root, char imerominia[11])
{
if (root == NULL || strcmp(root->date,imerominia)==0)
return root;
```

```
if (strcmp(root->date,imerominia)<0)
return search(root->right, imerominia);
return search(root->left, imerominia);
}

void update(struct Node* root, char imerominia[11], float nea_thermokrasia)

{
struct Node *komvos = search(root,imerominia);
if(komvos!=NULL)
{
komvos->temperature= nea_thermokrasia;
printf("Update Done\n");

}
else
{
printf("Node Not Found\n");
}
}

int main()
{
struct Node *root = NULL;
mydata array[1406];
FILE *fp;
```

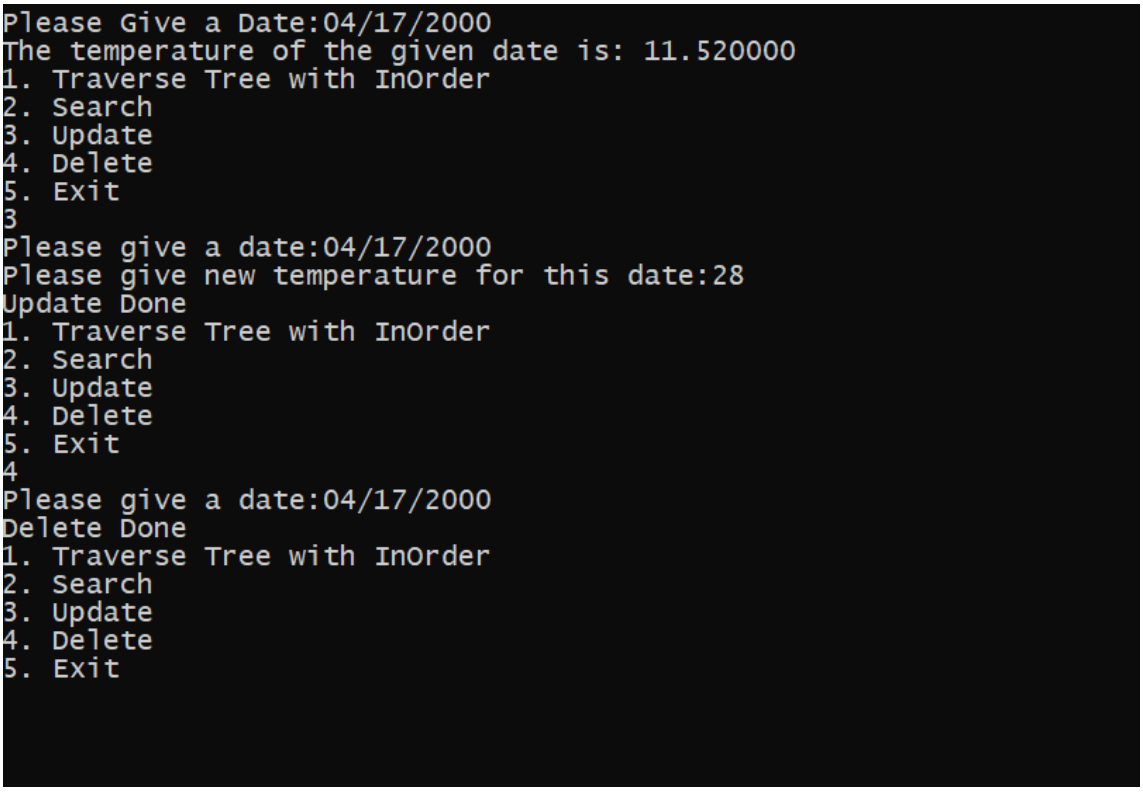
```
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
int i;
for(i=0;i<count;i++)
{
root = insert(root, array[i].date, array[i].temperature);

}
printf("Data loaded to tree successfully\n");
int selection;
printf("1. Traverse Tree with InOrder\n");
printf("2. Search\n");
printf("3. Update\n");
printf("4. Delete\n");
```

```
printf("5. Exit\n");
scanf("%d",&selection);
while(selection!=5)
{
if(selection==1)
{
printf("Results Of Traverse with InOrder\n");
printInorder(root);

}
else if(selection==2)
{
char given[11];
printf("Please Give a Date:");
scanf("%s",given);
struct Node* result = search(root, given);
if(result!=NULL)
{
printf("The temperature of the given date is: %f\n",result->temperature);
}
else
{
printf("Node Not Found\n");
}
}
else if(selection==3)
```

```
{
char given_upd[11];
float nea_therm;
printf("Please give a date:");
scanf("%s",given_upd);
printf("Please give new temperature for this date:");
scanf("%f",&nea_therm);
update(root,given_upd,nea_therm);
}
else if(selection==4)
{
char given_del[11];
printf("Please give a date:");
scanf("%s", given_del);

root = deleteNode(root, given_del);
printf("Delete Done\n");
}
printf("1. Traverse Tree with InOrder\n");
printf("2. Search\n");
printf("3. Update\n");
printf("4. Delete\n");
printf("5. Exit\n");
scanf("%d",&selection);
}
return 0;
```

```
}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:



ΕΡΩΤΗΜΑ Β

Κώδικας που υλοποιήθηκε:

```
#include<stdio.h>
#include<stdlib.h>
#include <string.h>

typedef struct dedomena

{
char date[11];
float temperature;
} mydata;
struct Node
{
char date[11];
float temperature;
struct Node *left;
struct Node *right;
int height;
};

int max(int a, int b);
int height(struct Node *N)
{
if (N == NULL)
```

```
return 0;
return N->height;
}

int max(int a, int b)
{
return (a > b)? a : b;
}

struct Node* newNode(char imerominia[11], float thermokrasia)
{
struct Node* node = (struct Node*)
malloc(sizeof(struct Node));
strcpy(node->date,imerominia);
node->temperature = thermokrasia;
node->left = NULL;
node->right = NULL;
node->height = 1;
return(node);
}

struct Node *rightRotate(struct Node *y)
{
struct Node *x = y->left;
struct Node *T2 = x->right;
x->right = y;
```

```
y->left = T2;
y->height = max(height(y->left), height(y->right))+1;
x->height = max(height(x->left), height(x->right))+1;
return x;
}

struct Node *leftRotate(struct Node *x)
{
struct Node *y = x->right;
struct Node *T2 = y->left;
y->left = x;
x->right = T2;
x->height = max(height(x->left), height(x->right))+1;
y->height = max(height(y->left), height(y->right))+1;
return y;
}

int getBalance(struct Node *N)
{
if (N == NULL)
return 0;
return height(N->left) - height(N->right);
}

struct Node* insert(struct Node* node, char imer[11], float key)
{
```

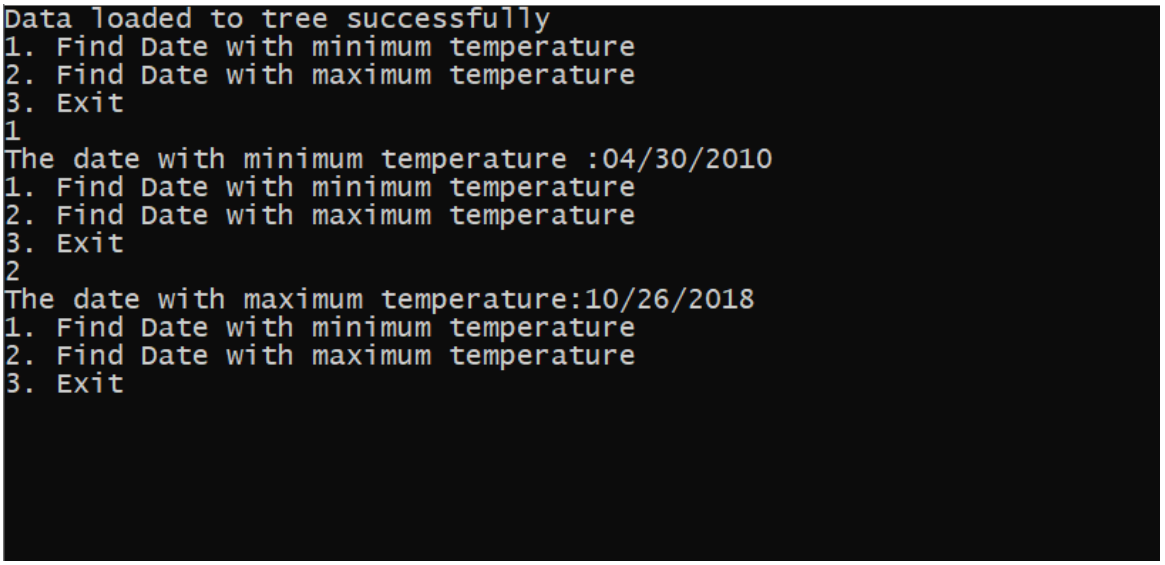
```
if (node == NULL)
return(newNode(imer, key));
if (key < node->temperature)
node->left = insert(node->left,imer, key);
else if (key > node->temperature)
node->right = insert(node->right,imer, key);
else
return node;
node->height = 1 + max(height(node->left),
height(node->right));
int balance = getBalance(node);
if (balance > 1 && key < node->left->temperature)
return rightRotate(node);
if (balance < -1 && key > node->right->temperature)
return leftRotate(node);
if (balance > 1 && key > node->left->temperature)
{
node->left = leftRotate(node->left);
return rightRotate(node);
}
if (balance < -1 && key < node->right->temperature)
{
node->right = rightRotate(node->right);
return leftRotate(node);
}
return node;
```

```
}

char* minValue(struct Node* node) {
struct Node* current = node;

while (current->left != NULL) {
current = current->left;
}
return(current->date);
}

char* maxValue(struct Node* node) {
struct Node* current = node;

while (current->right != NULL) {
current = current->right;
}
return(current->date);
}

int main()

{
struct Node *root = NULL;
mydata array[1406];
FILE *fp;
```

```
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
int i;
for(i=0;i<count;i++)
{
root = insert(root, array[i].date, array[i].temperature);

}
printf("Data loaded to tree successfully\n");
int selection;
printf("1. Find Date with minimum temperature\n");
printf("2. Find Date with maximum temperature\n");
printf("3. Exit\n");
scanf("%d",&selection);
```

```
while(selection!=3)
{
if(selection==1)
{
printf("The date with minimum temperature :%s\n",minValue(root));
}
else if(selection==2)
{
printf("The date with maximum temperature:%s\n",maxValue(root));
}
printf("1. Find Date with minimum temperature\n");
printf("2. Find Date with maximum temperature\n");
printf("3. Exit\n");
scanf("%d",&selection);
}
}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:



ΕΡΩΤΗΜΑ Γ

Κώδικας που υλοποιήθηκε:

```
#include<stdio.h>
#include<stdlib.h>

#define size 11

typedef struct dedomena

{
char date[11];
float temperature;
} mydata;

struct node
{
float temperature;
char date[11];
struct node *next;
};

struct node *chain[size];

void initialize_table()
{
```

```
int i;
for(i = 0; i < size; i++)
chain[i] = NULL;
}

int calculate_hash_value(char str[11]);

void insert(char imerominia[11],float thermokrasia)
{
struct node *newNode = malloc(sizeof(struct node));
strcpy(newNode->date, imerominia);
newNode->temperature= thermokrasia;
newNode->next = NULL;

int key = calculate_hash_value(imerominia);

if(chain[key] == NULL)
chain[key] = newNode;
else
{
struct node *temp = chain[key];
while(temp->next)
{
temp = temp->next;
}
```

```
temp->next = newNode;
}
}

void print()
{
int i;

for(i = 0; i < size; i++)
{
struct node *temp = chain[i];
printf("table[%d]-->",i);
while(temp)
{
printf("(%s ,%d) -->",temp->date, temp->temperature);
temp = temp->next;
}
printf("NULL\n");
}
}

int calculate_hash_value(char str[11])

{
int sum=0;
int i;
```

```
for(i=0;i<10;i++)
{
sum = sum + str[i];
}
int key = sum %size;
return key;
}

float search(char imerominia[11])
{
int key = calculate_hash_value(imerominia);
struct node *temp = chain[key];
while(temp)
{
if(strcmp(temp->date,imerominia)==0)
return temp->temperature;
temp = temp->next;
}
return 0;
}

void update(char imerominia[11], int nea_thermokrasia)



{
int key = calculate_hash_value(imerominia);
```

```
struct node *temp = chain[key];
while(temp)
{
if(strcmp(temp->date,imerominia)==0)
temp->temperature=nea_thermokrasia;
temp = temp->next;
}
}

int delete_node(char imerominia[11])
{
int key = calculate_hash_value(imerominia);
struct node *temp = chain[key], *dealloc;
if(temp != NULL)
{
if(strcmp(temp->date,imerominia)==0)
{
dealloc = temp;
temp = temp->next;
free(dealloc);
return 1;
}
else
{
while(temp->next)
{
```

```
if(strcmp(temp->next->date,imerominia)==0)
{
dealloc = temp->next;
temp->next = temp->next->next;
free(dealloc);
return 1;
}
temp = temp->next;
}
}
}

return 0;
}



int main()
{
initialize_table();
mydata array[1406];
FILE *fp;
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
```

```
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
int i;
for(i=0;i<count;i++)
{
insert(array[i].date, array[i].temperature);

}
printf("Data loaded to Hash Table successfully\n");



int selection;
printf("1. Search\n");
printf("2. Update\n");
printf("3. Delete\n");
printf("4. Exit\n");
scanf("%d",&selection);
while(selection!=4)
{
```

```
if(selection==1)
{
printf("Please give date to search:");
char given_search[11];
scanf("%s",given_search);
printf("The temperature for the given Date is:%f\n",search(given_search));
}
else if(selection==2)
{
char given_upd[11];
float nea_therm;
printf("Please give date:");
scanf("%s",given_upd);
printf("Please give New Temperature:");
scanf("%f",&nea_therm);
update(given_upd,nea_therm);
}
else if(selection==3)
{
char given_del[11];
printf("Please give date:");
scanf("%s",given_del);
if(delete_node(given_del)==1)
{
printf("Delete Done successfully\n");
}
```

```
else
{
printf("Node Not Found\n");
}
}
printf("1. Search\n");
printf("2. Update\n");
printf("3. Delete\n");
printf("4. Exit\n");
scanf("%d",&selection);
}



return 0;
}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα:

```
Data loaded to Hash Table successfully
1. Search
2. Update
3. Delete
4. Exit
1
Please give date to search:01/07/2000
The temperature for the given Date is:11.000000
1. Search
2. Update
3. Delete
4. Exit
2
Please give date:01/07/2000
Please give New Temperature:36
1. Search
2. Update
3. Delete
4. Exit
3
Please give date:01/07/2000
Delete Done successfully
1. Search
2. Update
3. Delete
4. Exit
```

ΣΥΝΔΥΑΣΤΙΚΟ ΕΡΩΤΗΜΑ:

Κώδικας που υλοποιήθηκε:

```
#include<stdio.h>

#include<stdlib.h>

#include <string.h>

#define size 11



typedef struct dedomena
```

```
{
char date[11];
float temperature;
} mydata;
struct Node
{
char date[11];
float temperature;
struct Node *left;
struct Node *right;
int height;
};

struct node
{
float temperature;
char date[11];
struct node *next;
};
struct node *chain[size];

int max(int a, int b);

int height(struct Node *N)
{
```

```
if (N == NULL)
return 0;
return N->height;
}

int max(int a, int b)
{
return (a>b)? a : b;
}

struct Node* newNode(char imerominia[11], float thermokrasia)
{
struct Node* node = (struct Node*)malloc(sizeof(struct Node));
strcpy(node->date,imerominia);
node->temperature = thermokrasia;
node->left = NULL;
node->right = NULL;
node->height = 1;
return(node);
}

struct Node *rightRotate(struct Node *y)
{
struct Node *x = y->left;
struct Node *T2 = x->right;
x->right = y;
```

```
y->left = T2;


y->height = max(height(y->left), height(y->right))+1;
x->height = max(height(x->left), height(x->right))+1;
return x;
}


struct Node *leftRotate(struct Node *x)
{
struct Node *y = x->right;
struct Node *T2 = y->left;
y->left = x;
x->right = T2;
x->height = max(height(x->left), height(x->right))+1;
y->height = max(height(y->left), height(y->right))+1;
return y;
}
int getBalance(struct Node *N)
{
if (N == NULL)
return 0;
return height(N->left) - height(N->right);
}


struct Node* insert_to_avl_by_date(struct Node* node, char imerominia[11], float
thermokrasia)
{
```

```
if (node == NULL)
return(newNode(imerominia,thermokrasia));
if (strcmp(imerominia,node->date)<0)
node->left = insert_to_avl_by_date(node->left, imerominia,thermokrasia);
else if (strcmp(imerominia,node->date)>0)
node->right = insert_to_avl_by_date(node->right, imerominia,thermokrasia);
else
return node;
node->height = 1 + max(height(node->left),
height(node->right));

int balance = getBalance(node);
if (balance > 1 && strcmp(imerominia, node->left->date)<0)
return rightRotate(node);
if (balance < -1 && strcmp(imerominia, node->right->date)>0)
return leftRotate(node);
if (balance > 1 && strcmp(imerominia, node->left->date)>0)
{
node->left = leftRotate(node->left);
return rightRotate(node);
}
if (balance < -1 && strcmp(imerominia, node->right->date)<0)
{
node->right = rightRotate(node->right);
return leftRotate(node);
}
```

```
return node;
}



struct Node* insert_to_avl_by_temperature(struct Node* node, char imer[11], float
key)
{
if (node == NULL)
return(newNode(imer, key));
if (key < node->temperature)
node->left = insert_to_avl_by_temperature(node->left,imer, key);
else if (key > node->temperature)
node->right = insert_to_avl_by_temperature(node->right,imer, key);
else
return node;
node->height = 1 + max(height(node->left),
height(node->right));
int balance = getBalance(node);
if (balance > 1 && key < node->left->temperature)
return rightRotate(node);
if (balance < -1 && key > node->right->temperature)
return leftRotate(node);
if (balance > 1 && key > node->left->temperature)
{
node->left = leftRotate(node->left);
return rightRotate(node);
}
```

```
if (balance < -1 && key < node->right->temperature)
{
node->right = rightRotate(node->right);
return leftRotate(node);
}
return node;
}

void initialize_table()
{
int i;
for(i = 0; i < size; i++)
chain[i] = NULL;
}

int calculate_hash_value(char str[11])

{
int sum=0;
int i;
for(i=0;i<10;i++)
{
sum = sum + str[i];
}
int key = sum %size;
return key;
```

```
}

void insert_to_chain(char imerominia[11],float thermokrasia)
{
struct node *newNode = malloc(sizeof(struct Node));
strcpy(newNode->date, imerominia);
newNode->temperature= thermokrasia;
newNode->next = NULL;

int key = calculate_hash_value(imerominia);

if(chain[key] == NULL)
chain[key] = newNode;
else
{
struct node *temp = chain[key];
while(temp->next)
{
temp = temp->next;
}

temp->next = newNode;
}
}

int main()
```

```
{
struct Node *root = NULL;
struct node *root_chain = NULL;
mydata array[1406];
FILE *fp;
fp = fopen("ocean.csv", "r");
char line[200];
char *token;
int count=0;
fscanf(fp,"%s\n",line);
while(fscanf(fp,"%s\n",line)!=EOF)
{
token = strtok(line, ",");
strcpy(array[count].date,token);
token = strtok(NULL, ",");
array[count].temperature = atof(token);
count++;

}
int i;
printf("Welcome!Please select one of the following choices:\n");
printf("1. AVL\n");
printf("2.Hasing with chains\n");
int selection;
scanf("%d",&selection);
```

```
if(selection==1)
{
printf("1.Create AVL by Date\n");
printf("2.Create AVL by Temperature\n");
int choice;
scanf("%d",&choice);
if(choice==1)
{
for(i=0;i<count;i++)
{
root = insert_to_avl_by_date(root,array[i].date, array[i].temperature);

}
printf("Data Loaded!!!!");
}
else if (choice==2)
{
for(i=0;i<count;i++)
{
root = insert_to_avl_by_temperature(root, array[i].date, array[i].temperature);

}
printf("Data Loaded!!!!");

}
}
```

```
else if(selection==2)

{

initialize_table();


for(i=0;i<count;i++)

{

insert_to_chain(array[i].date, array[i].temperature);


}

printf("Data Loaded!!!!");

}

}
```

Αποτέλεσμα μετά το πέρας της εκτέλεσης του κώδικα: